**Universidade Estadual de Maringá (UEM)**
**Departamento de Informática (DIN)**

| | | | |
|---|---|---|---|
| **Curso:** | | | |
| **Disciplina:** | Programação de Sistemas Web | | |
| **Professor:** | Lucas de Oliveira Teixeira | **Data:** | _____ |
| **Aluno:** | _____ | **R.A.:** | _____ |

# ANP 10: Marco 2

**Quando fazer:** após a Aula 4 do módulo de Node (fechamento do Marco 2).
**Tempo estimado:** 60 a 120 minutos.
**Entrega:** commits de validação e documentação + `docs/api.md` + `docs/testes-parte2.md`; tag `v2.0` apontando para a versão final.

## Contexto

A Aula 4 integrou frontend e backend e fechou o Marco 2 com a tag `v2.0`. Esta ANP é o polimento final do CineTrack: endurecer a validação do servidor (que hoje aceita nota 7 e status inventado), documentar a API para quem for consumi-la e provar, com um teste ponta a ponta, que o sistema inteiro funciona a partir de um clone limpo.

**Você parte de:** o CineTrack full stack da Aula 4: CRUD completo pela API, CORS, `.env` e a tag `v2.0`. Esta ANP corresponde ao **Marco 2** do trabalho.

## Tarefas

1. Endureça a validação de `POST /filmes` e `PUT /filmes/:id`: além de título e ano, exija `nota` entre 1 e 5 e `status` entre `assistido`, `assistindo` e `quero`. A resposta `400` devolve **todos** os problemas de uma vez: `{ "erros": ["nota deve estar entre 1 e 5", ...] }`; porte a lógica de `validarFilme` da Parte 1.
2. Teste com um cliente HTTP: um `POST` com nota 7 **e** status inválido deve devolver 400 com os dois erros na lista. Confira no frontend que o estado de erro mostra algo útil ao usuário.
3. Crie `docs/api.md` com uma tabela por rota: método, caminho, parâmetros ou corpo esperado e códigos de resposta possíveis (200, 201, 204, 400, 404).
4. Teste ponta a ponta a partir do zero, registrando cada passo em `docs/testes-parte2.md`: `node seed.js`, subir o backend, subir o frontend, executar o CRUD completo pela interface, reiniciar o servidor e conferir que os dados persistiram.
5. Revise o `README.md` final e, se houve correções depois da tag, mova-a: `git tag -f v2.0` e `git push -tags -force`.

## Critérios de aceitação

- `POST` com nota 7 e status inválido devolve 400 com a **lista** dos dois erros; dado válido continua devolvendo 201.
- O `PUT` aplica a mesma validação do `POST`.
- `docs/api.md` cobre todas as rotas do servidor, com códigos de resposta corretos.
- `docs/testes-parte2.md` registra o teste ponta a ponta com resultados reais, incluindo a persistência após reiniciar.
- A tag `v2.0` aponta para a versão final; o `README` ensina a rodar backend e frontend do zero.

## Onde pesquisar

- MDN, códigos de status HTTP: https://developer.mozilla.org/pt-BR/docs/Web/HTTP/Status